

The background features a dark blue gradient on the left, transitioning into a large, vibrant, abstract shape on the right. This shape is composed of overlapping curved segments in shades of orange, pink, and purple, creating a dynamic, modern aesthetic.

AWS re:Invent

NOV. 27 – DEC. 1, 2023 | LAS VEGAS, NV

SAS405 - R

Federating identity in multi-tenant SaaS environments

Toby Buckley

(he/him)

Sr. Partner Solutions Architect,
AWS SaaS Factory

AWS

Dhammika Sriyananda

(he/him)

Sr. Partner Solutions Architect,
AWS SaaS Factory

AWS



Agenda

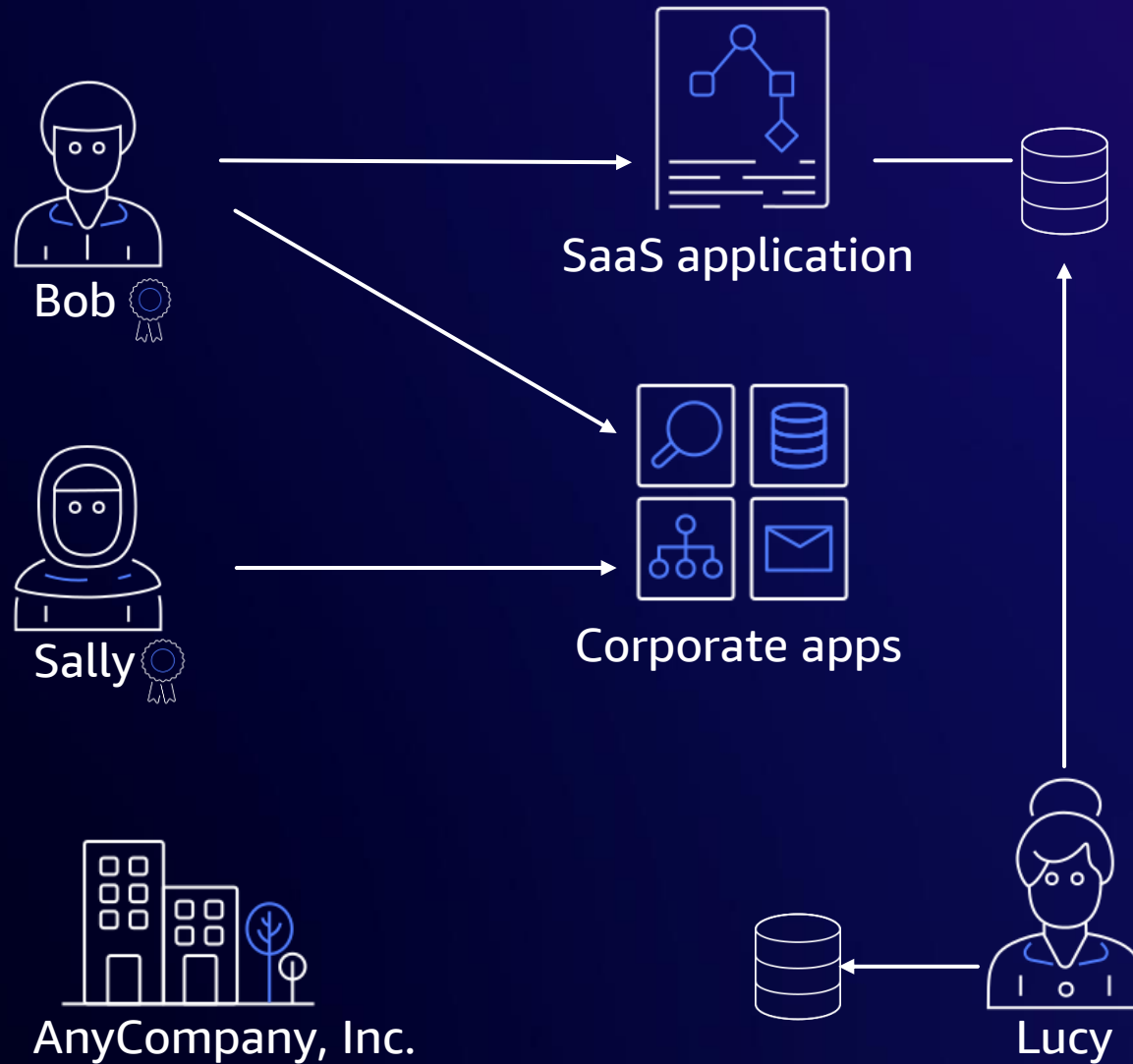
- Q&A
- We describe the problem we see (unfederated SaaS)
- We propose a solution – and AWS services that help
- What new problems arise?
 - We offer some suggestions
- Q&A

Q&A

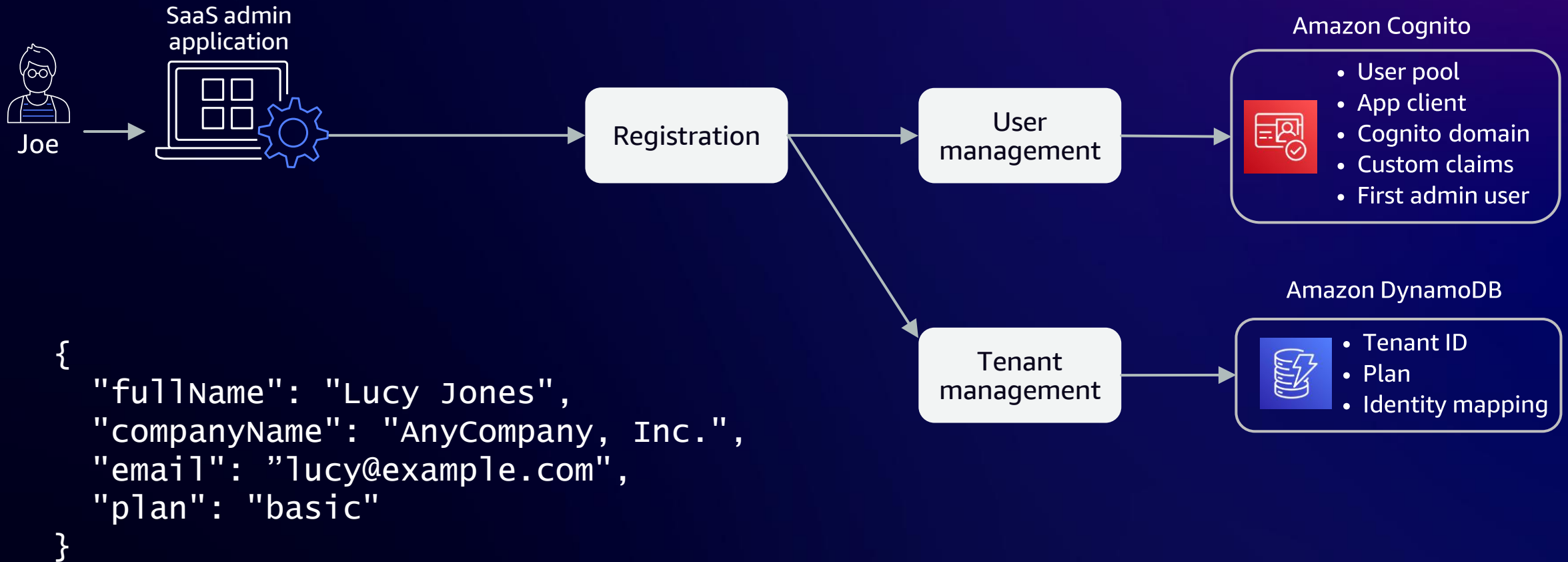
By a show of hands, are you here because ...

1. You're currently operating a SaaS solution?
2. You're actively building a SaaS solution?
3. You're planning or designing SaaS?
4. You had nothing better to do?

Unfederated SaaS



Unfederated identity onboarding



Unfederated model

- Create the user pool with custom attributes (tenant context)
- Create the first admin user – That user has CRUD authority over additional users
- Admin user creates additional users
- App resolves which user pool to use, and Amazon Cognito issues tokens
- Authorization based on authenticated user + claims

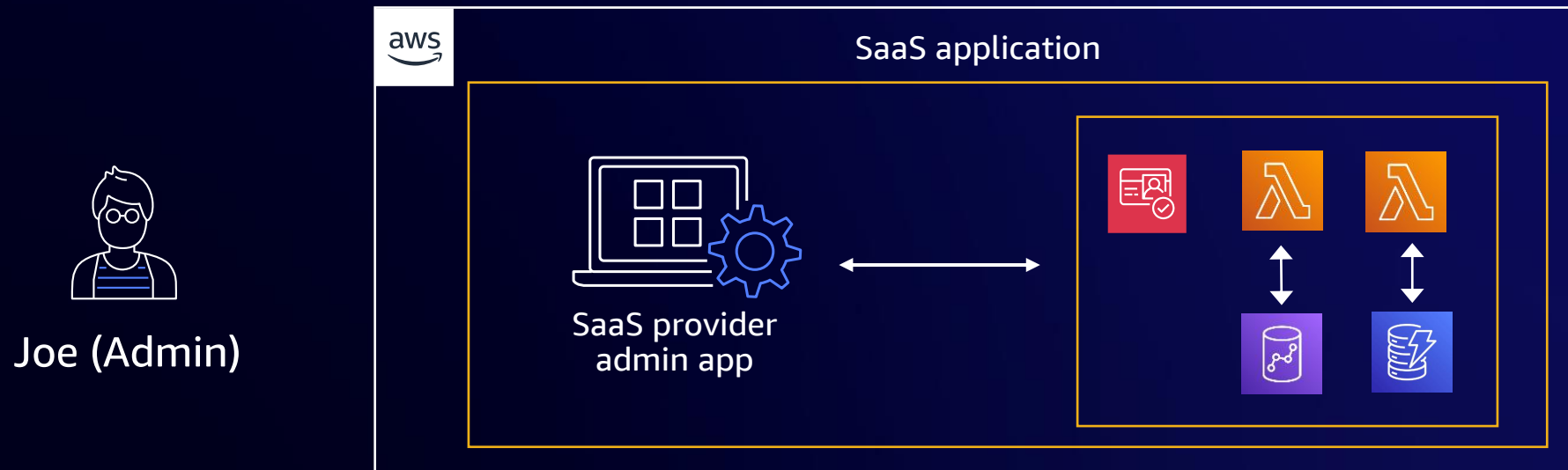
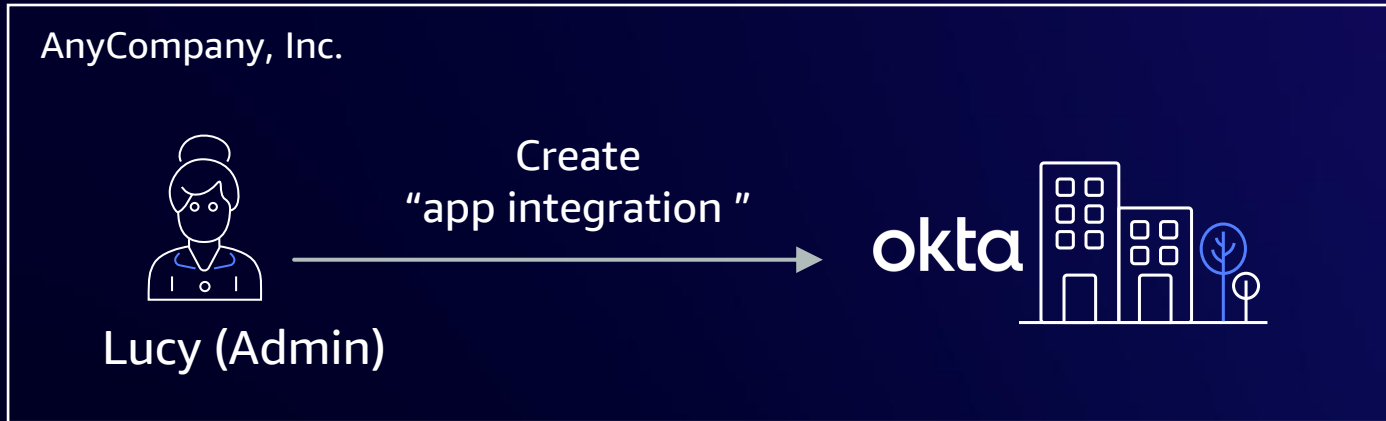
Enter federation



Q&A

- Is this why you're here?
- Anything specific we *didn't* mention?

New tenant onboarding with federated identity

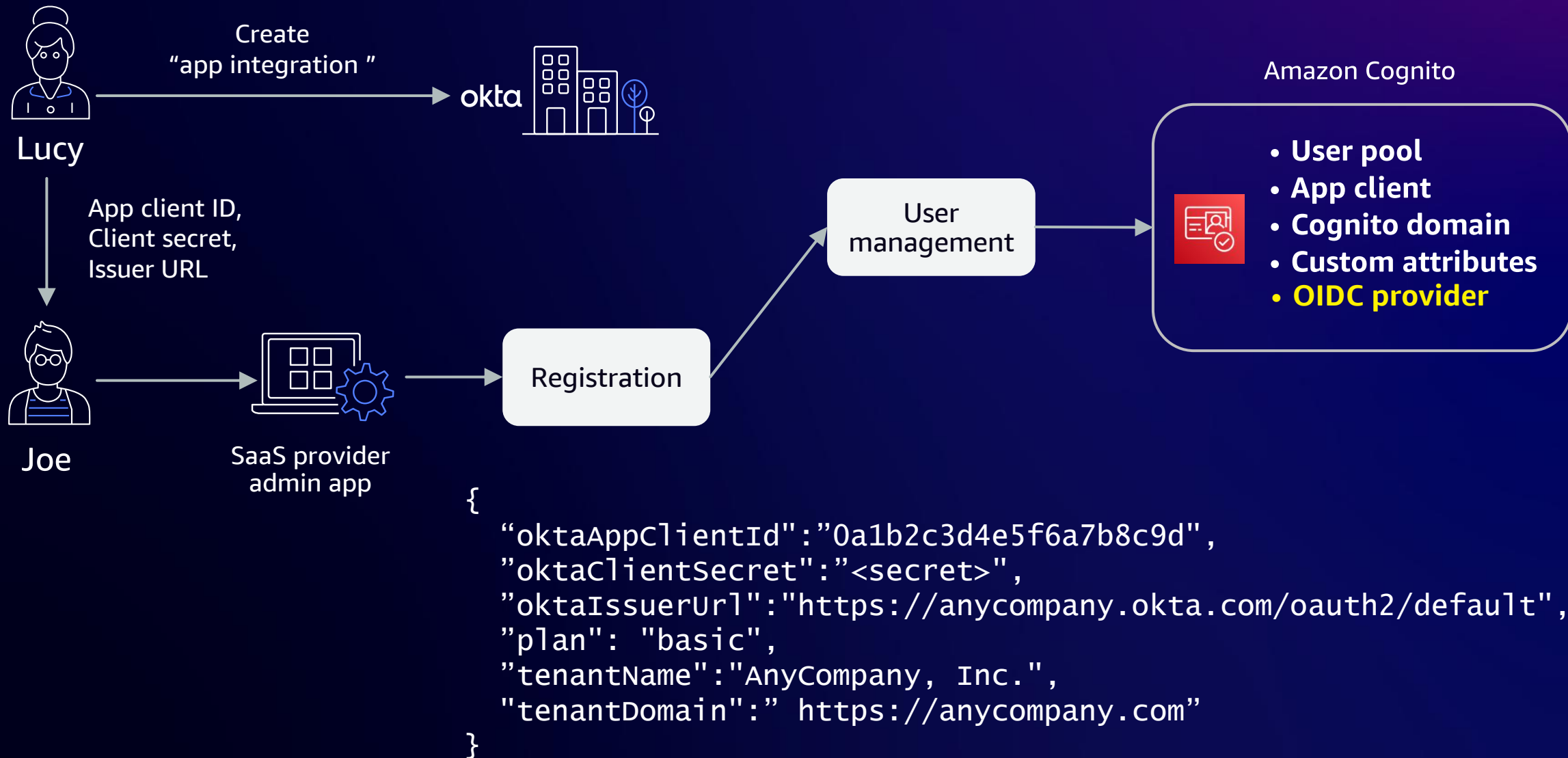


Okta console – Attribute mapping

Username is set by [Cognito-Test](#) · [Override with mapping](#)

user.firstName	▼	→ ▼	userName
user.nickName	▼	→ ▼	name
user.firstName	▼	→ ▼	nickname
user.firstName	▼	→ ▼	given_name
user.middleName	▼	→ ▼	middle_name
user.lastName	▼	→ ▼	family_name
user.email	▼	→ ▼	email
user.profileUrl	▼	→ ▼	profile

New tenant onboarding with federated identity



Amazon Cognito - OIDC provider configs

Identity provider information

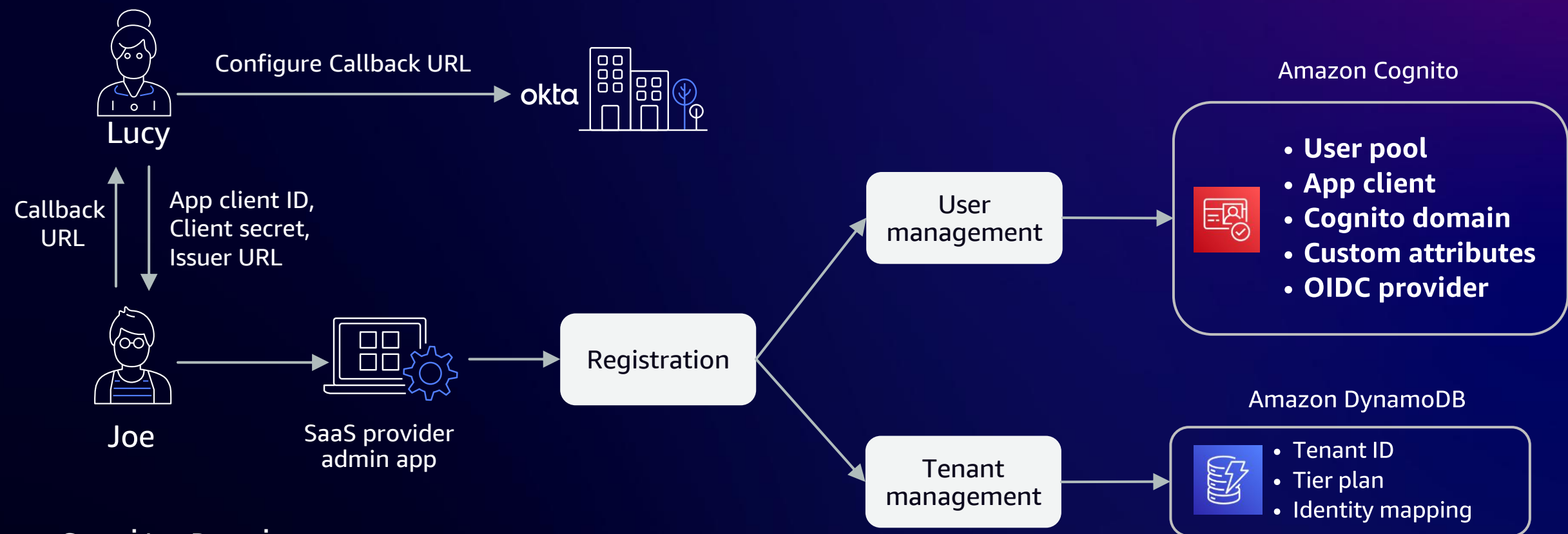
Provider type OIDC	Identifiers -
Provider name oktaoidc	Attribute request method GET
Client ID 7asdna6az86hbOsdNHGIDD5d7	Issuer https://anycompany.okta.com/oauth2/default
Client secret assdg87YNKSJ7T_30W1j0vOYslsgweggsoc8bngoAiasdfds Nyww2Refnl-_qa	Authorization endpoint -

Attribute mapping (3) [Info](#)

View, add, and edit attribute mappings between OpenID Connect and your user pool.

User pool attribute	OpenID Connect attribute
email	email
name	name
username	sub

New tenant onboarding with federated identity



Cognito Domain :
"https://anycompanyoidc.auth.us-east-1.amazonaws.com"

```
{  
  "CallbackURL": "<cognito-domain>/oauth2/idpresponse"  
}
```

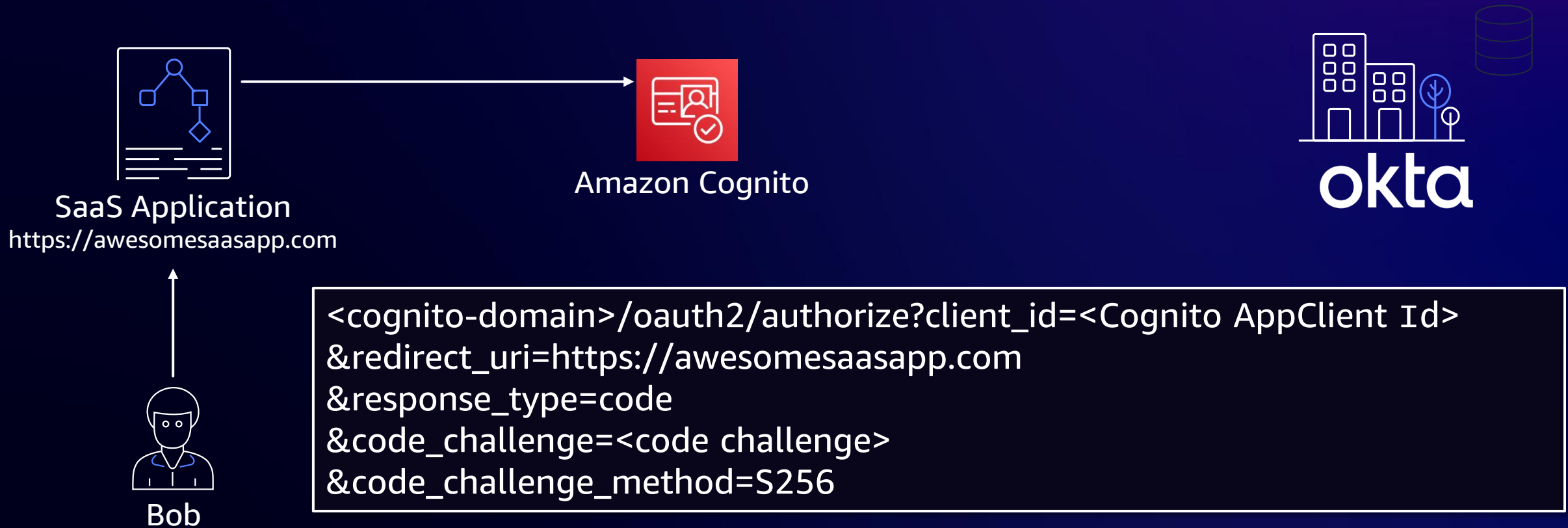
Company identifier	Tenant ID
anycompany.com	Tenant-003

Tenant management DB

Federated tenant user login – Entire flow



Federated tenant user login – Entire flow



Federated tenant user login – Entire flow



Federated tenant user login – Entire flow



Federated tenant user login – Entire flow



Federated tenant user login – Entire flow



Federated tenant user login – Entire flow



Federated tenant user login – Entire flow



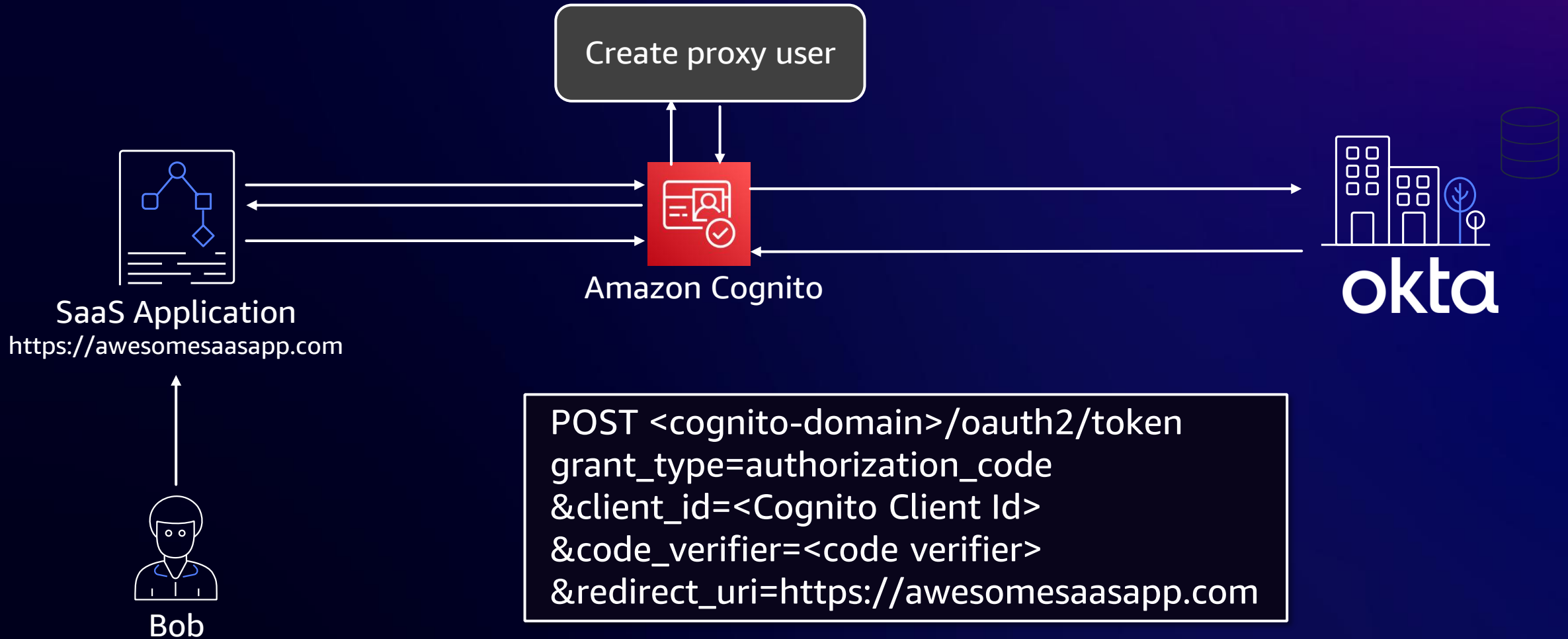
Federated tenant user login – Entire flow



Federated tenant user login – Entire flow



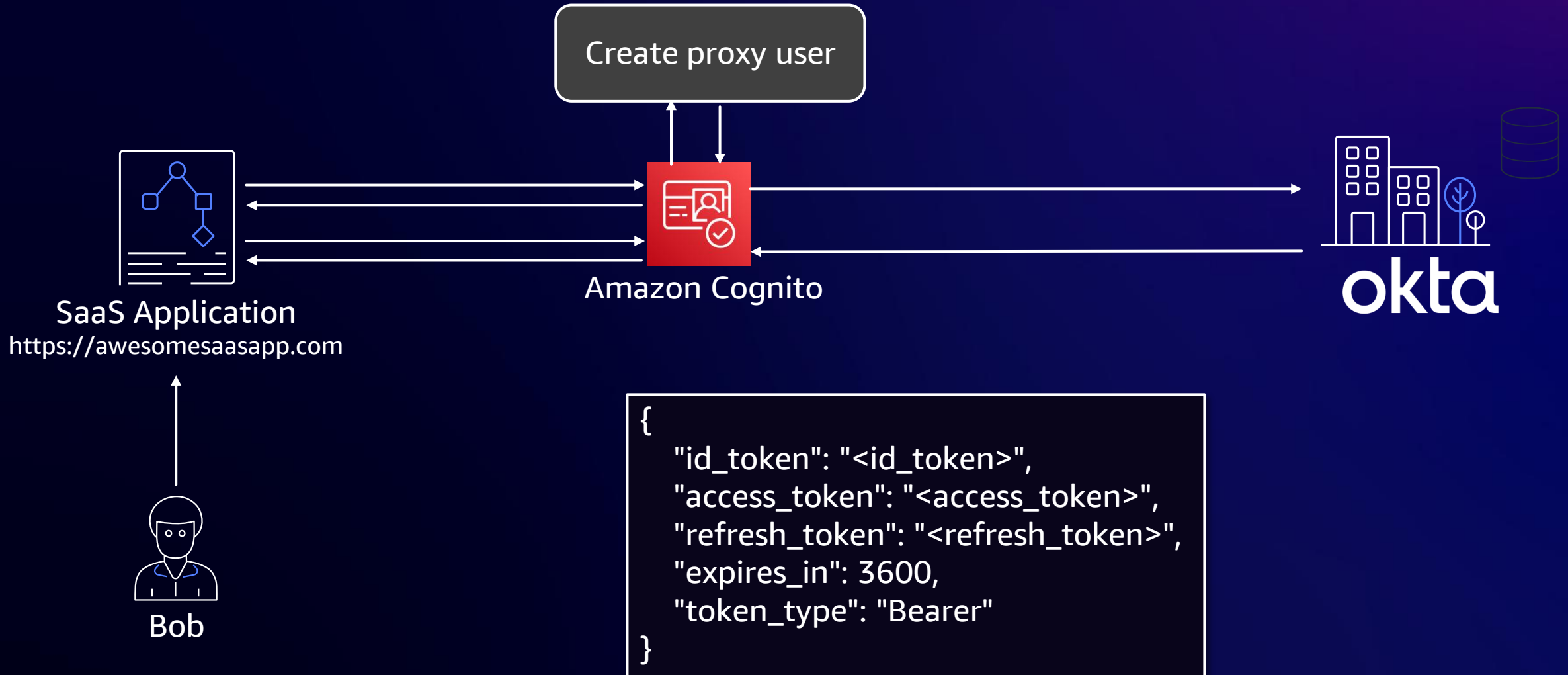
Federated tenant user login – Entire flow



Federated tenant user login – Entire flow



Federated tenant user login – Entire flow



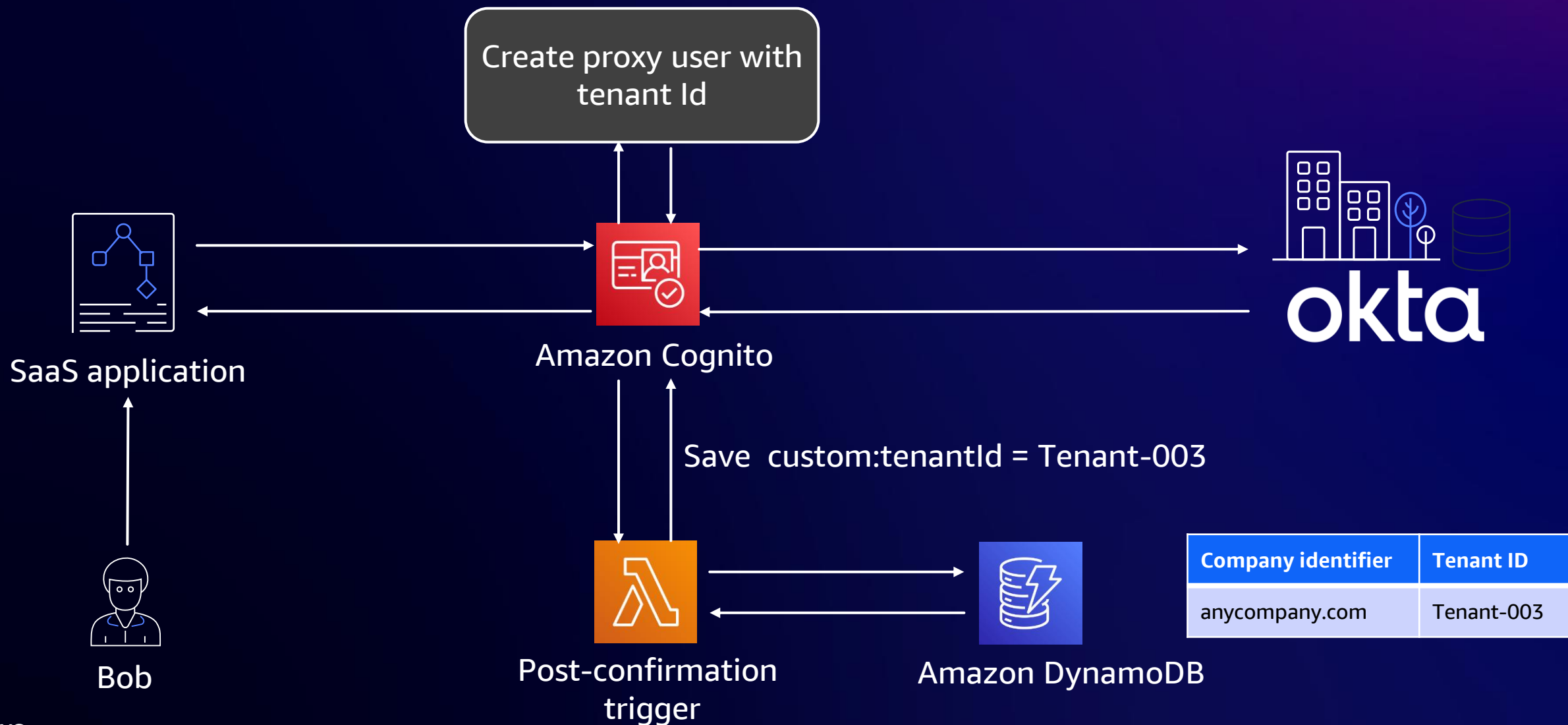
Q&A

- Any thoughts?
- Did we miss anything in the flow that affects tenant-wise user operations?

Inject the tenant ID – Cognito Lambda triggers

User Pool Flow	Operation
Custom Authentication Flow	Define Auth Challenge
	Create Auth Challenge
	Verify Auth Challenge Response
Authentication Events	Pre authentication Lambda trigger
	Post authentication Lambda trigger
	Pre token generation Lambda trigger
Sign-Up	Pre sign-up Lambda trigger
	Post confirmation Lambda trigger
	Migrate user Lambda trigger
Messages	Custom message Lambda trigger
Token Creation	Pre token generation Lambda trigger
Email and SMS third-party providers	Custom sender Lambda triggers

Adding the tenant context



Inside the Lambda trigger

```
1  export const handler = async (  
2    event: PostConfirmationTriggerEvent,  
3    context: Context  
4  ): Promise<any> => {  
5    const email = event.request.userAttributes.email;  
6    const tenantId = await DynamoDB.fetchTenantId(email);  
7    const client = new CognitoIdentityProviderClient({ region: process.env.AWS_REGION });  
8    const input = {  
9      UserPoolId: event.userPoolId,  
10     Username: event.userName,  
11     UserAttributes: [  
12       {  
13         Name: 'custom:tenant-id',  
14         Value: tenantId,  
15       },  
16     ],  
17   };  
18   const command = new AdminUpdateUserAttributesCommand(input);  
19   await client.send(command);  
20   return event;  
21 };
```



Company identifier	Tenant ID
amazon.com	Tenant-001
anycompany.com	Tenant-003

Inside the Lambda trigger

```
1  export const handler = async (  
2    event: PostConfirmationTriggerEvent,  
3    context: Context  
4  ): Promise<any> => {  
5    const email = event.request.userAttributes.email;  
6    const tenantId = await DynamoDB.fetchTenantId(email);  
7    const client = new CognitoIdentityProviderClient({ region: process.env.AWS_REGION });  
8    const input = {  
9      UserPoolId: event.userPoolId,  
10     Username: event.userName,  
11     UserAttributes: [  
12       {  
13         Name: 'custom:tenant-id',  
14         Value: tenantId,  
15       },  
16     ],  
17   };  
18   const command = new AdminUpdateUserAttributesCommand(input);  
19   await client.send(command);  
20   return event;  
21 };
```



Company identifier	Tenant ID
amazon.com	Tenant-001
anycompany.com	Tenant-003

Inside the Lambda trigger

```
1  export const handler = async (  
2    event: PostConfirmationTriggerEvent,  
3    context: Context  
4  ): Promise<any> => {  
5    const email = event.request.userAttributes.email;  
6    const tenantId = await DynamoDB.fetchTenantId(email);  
7    const client = new CognitoIdentityProviderClient({ region: process.env.AWS_REGION });  
8    const input = {  
9      UserPoolId: event.userPoolId,  
10     Username: event.userName,  
11     UserAttributes: [  
12       {  
13         Name: 'custom:tenant-id',  
14         Value: tenantId,  
15       },  
16     ],  
17   };  
18   const command = new AdminUpdateUserAttributesCommand(input);  
19   await client.send(command);  
20   return event;  
21 };
```



Company identifier	Tenant ID
amazon.com	Tenant-001
anycompany.com	Tenant-003

Inside the Lambda trigger

```
1  export const handler = async (  
2    event: PostConfirmationTriggerEvent,  
3    context: Context  
4  ): Promise<any> => {  
5    const email = event.request.userAttributes.email;  
6    const tenantId = await DynamoDB.fetchTenantId(email);  
7    const client = new CognitoIdentityProviderClient({ region: process.env.AWS_REGION });  
8    const input = {  
9      UserPoolId: event.userPoolId,  
10     Username: event.userName,  
11     UserAttributes: [  
12       {  
13         Name: 'custom:tenant-id',  
14         Value: tenantId,  
15       },  
16     ],  
17   };  
18   const command = new AdminUpdateUserAttributesCommand(input);  
19   await client.send(command);  
20   return event;  
21 };
```



Company identifier	Tenant ID
amazon.com	Tenant-001
anycompany.com	Tenant-003

Inside the Lambda trigger

```
1  export const handler = async (  
2    event: PostConfirmationTriggerEvent,  
3    context: Context  
4  ): Promise<any> => {  
5    const email = event.request.userAttributes.email;  
6    const tenantId = await DynamoDB.fetchTenantId(email);  
7    const client = new CognitoIdentityProviderClient({ region: process.env.AWS_REGION });  
8    const input = {  
9      UserPoolId: event.userPoolId,  
10     Username: event.userName,  
11     UserAttributes: [  
12       {  
13         Name: 'custom:tenant-id',  
14         Value: tenantId,  
15       },  
16     ],  
17   };  
18   const command = new AdminUpdateUserAttributesCommand(input);  
19   await client.send(command);  
20   return event;  
21 };
```



Company identifier	Tenant ID
amazon.com	Tenant-001
anycompany.com	Tenant-003

Inside the Lambda trigger

```
1  export const handler = async (  
2    event: PostConfirmationTriggerEvent,  
3    context: Context  
4  ): Promise<any> => {  
5    const email = event.request.userAttributes.email;  
6    const tenantId = await DynamoDB.fetchTenantId(email);  
7    const client = new CognitoIdentityProviderClient({ region: process.env.AWS_REGION });  
8    const input = {  
9      UserPoolId: event.userPoolId,  
10     Username: event.userName,  
11     UserAttributes: [  
12       {  
13         Name: 'custom:tenant-id',  
14         Value: tenantId,  
15       },  
16     ],  
17   };  
18   const command = new AdminUpdateUserAttributesCommand(input);  
19   await client.send(command);  
20   return event;  
21 };
```



Company identifier	Tenant ID
amazon.com	Tenant-001
anycompany.com	Tenant-003

Inside the Lambda trigger

```
1  export const handler = async (  
2    event: PostConfirmationTriggerEvent,  
3    context: Context  
4  ): Promise<any> => {  
5    const email = event.request.userAttributes.email;  
6    const tenantId = await DynamoDB.fetchTenantId(email);  
7    const client = new CognitoIdentityProviderClient({ region: process.env.AWS_REGION });  
8    const input = {  
9      UserPoolId: event.userPoolId,  
10     Username: event.userName,  
11     UserAttributes: [  
12       {  
13         Name: 'custom:tenant-id',  
14         Value: tenantId,  
15       },  
16     ],  
17   };  
18   const command = new AdminUpdateUserAttributesCommand(input);  
19   await client.send(command);  
20   return event;  
21 };
```



Company identifier	Tenant ID
amazon.com	Tenant-001
anycompany.com	Tenant-003

Inside the Lambda trigger

```
1  export const handler = async (  
2    event: PostConfirmationTriggerEvent,  
3    context: Context  
4  ): Promise<any> => {  
5    const email = event.request.userAttributes.email;  
6    const tenantId = await DynamoDB.fetchTenantId(email);  
7    const client = new CognitoIdentityProviderClient({ region: process.env.AWS_REGION });  
8    const input = {  
9      UserPoolId: event.userPoolId,  
10     Username: event.userName,  
11     UserAttributes: [  
12       {  
13         Name: 'custom:tenant-id',  
14         Value: tenantId,  
15       },  
16     ],  
17   };  
18   const command = new AdminUpdateUserAttributesCommand(input);  
19   await client.send(command);  
20   return event;  
21 };
```



Company identifier	Tenant ID
amazon.com	Tenant-001
anycompany.com	Tenant-003

Amazon Cognito user pool – Federated user attributes

Attribute name ▲	Value
custom:tenant-id	Tenant-003
email	bob@anycompany.com Not verified
family_name	
given_name	Bob
identities	[{"userId":"00u9zk52xj8pF0VeT5d7","providerName":"OktaOIDC","providerType":"OIDC","issuer":null,"primary":true,"dateCreated":1686874550709}]
name	Bob
sub	e5fd1f94-e0ad-425f-a414-1368d4b4b942

Group memberships (1) [Info](#)

View and edit this user's group memberships.

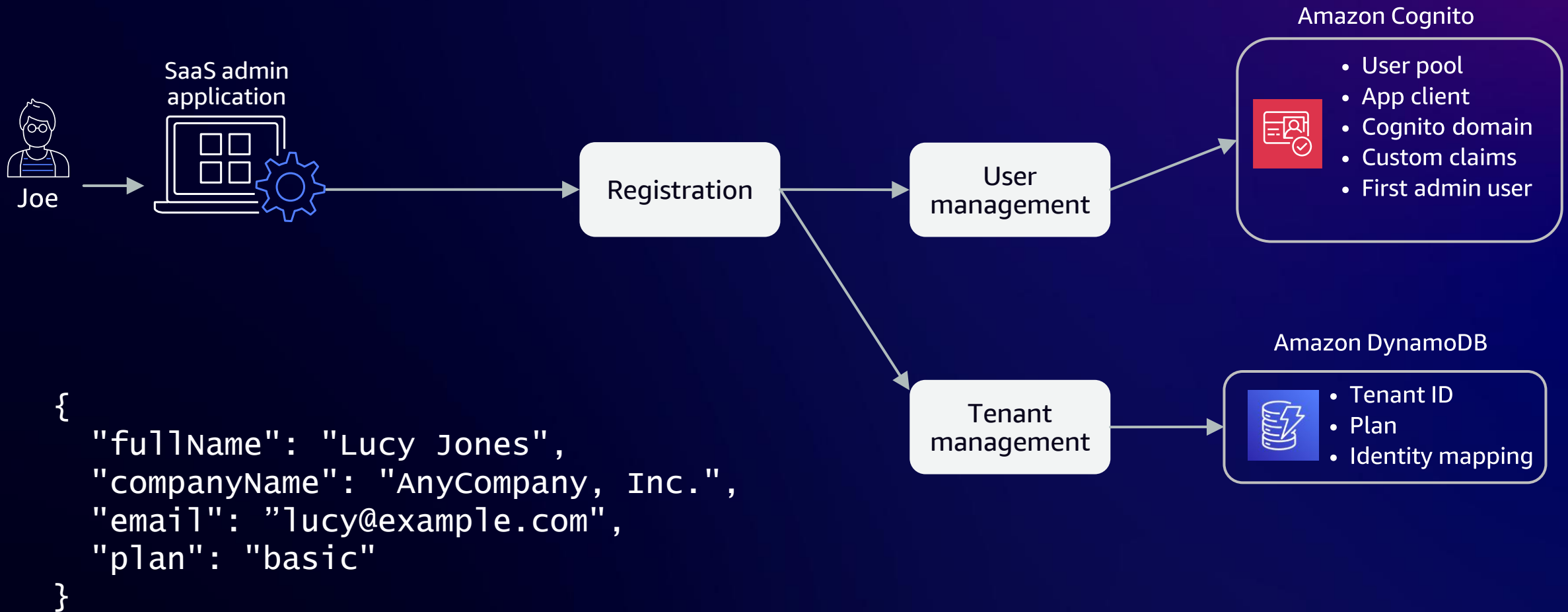
Group name ▲	Description ▼
☐ us-west-2_MbvFeynyu_OktaOIDC	Autogenerated group for users who sign in using OktaOIDC

[Remove user from group](#)

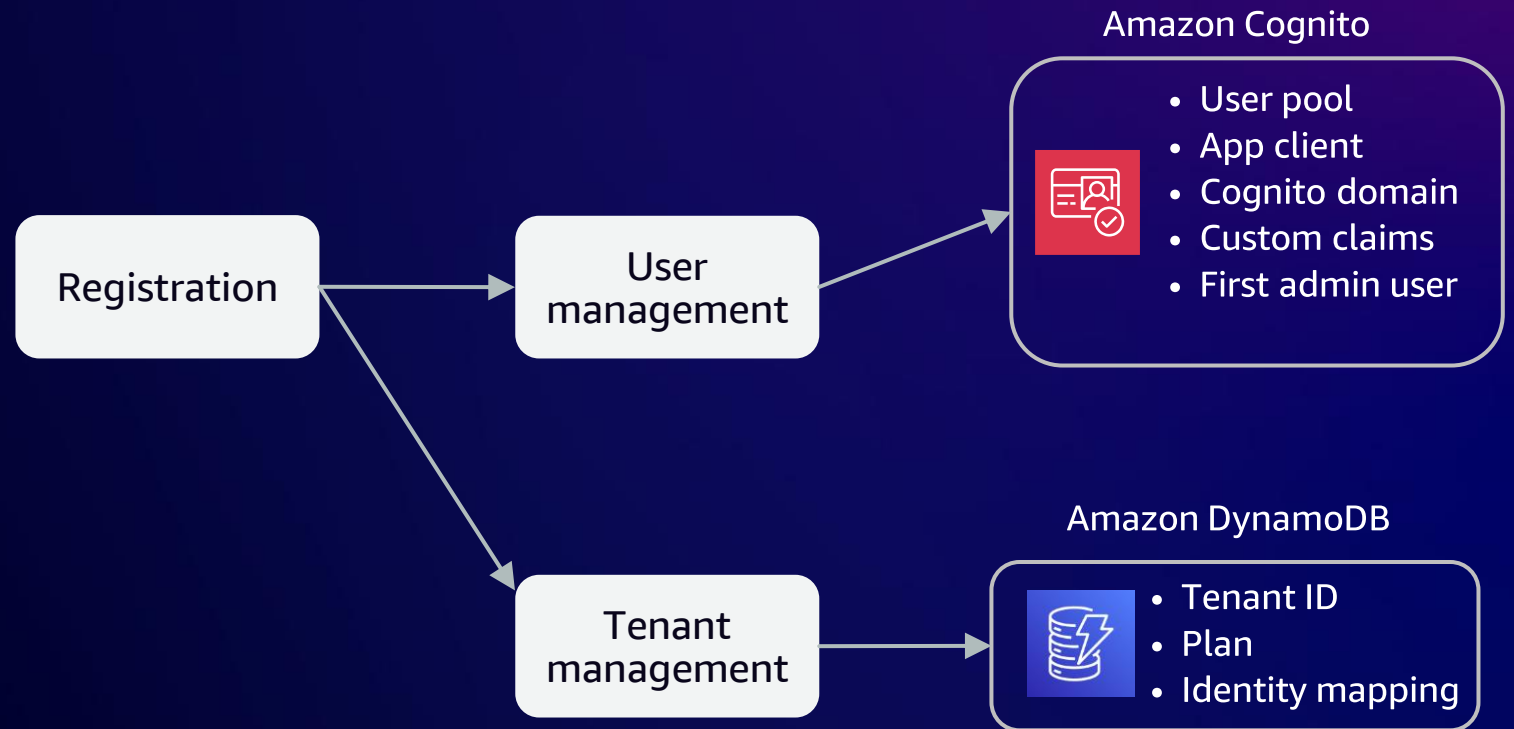
Q&A

- How could we further optimize this flow with federation?
- A more automated, self-service tenant onboarding flow with federation?

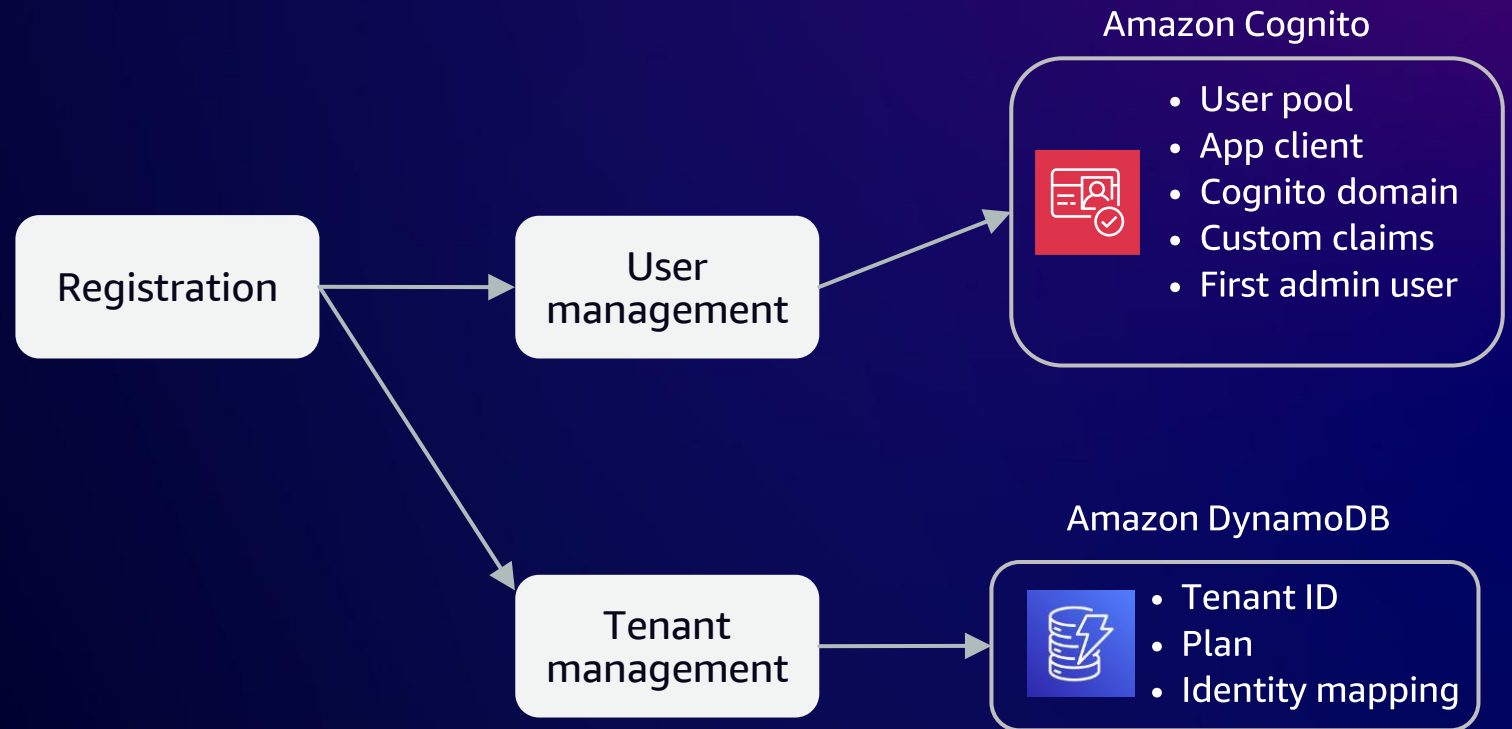
Tenant onboarding automation with federation



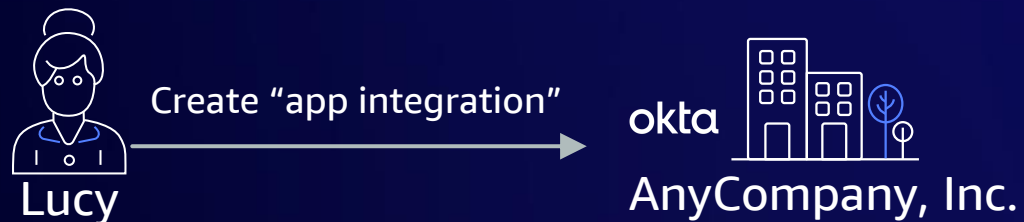
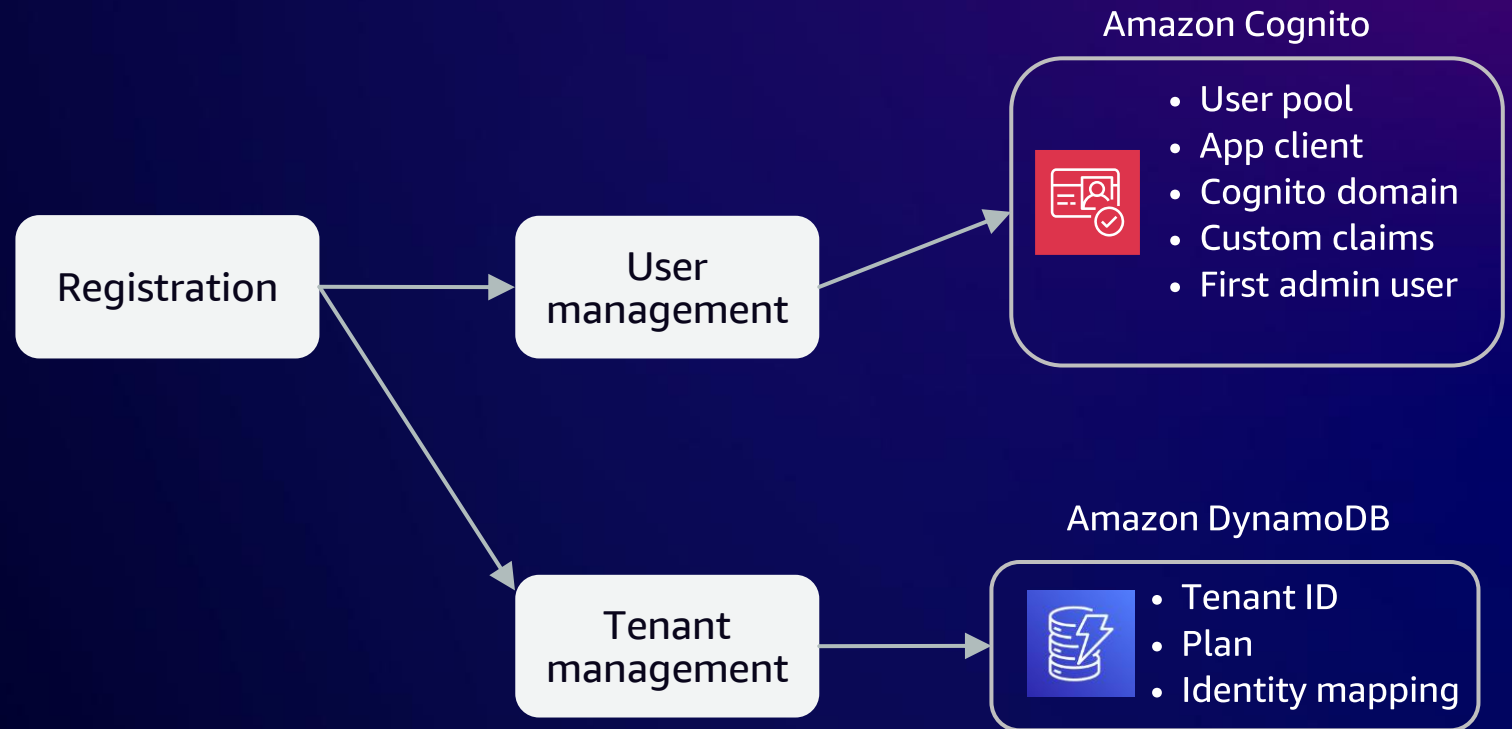
Tenant onboarding automation with federation



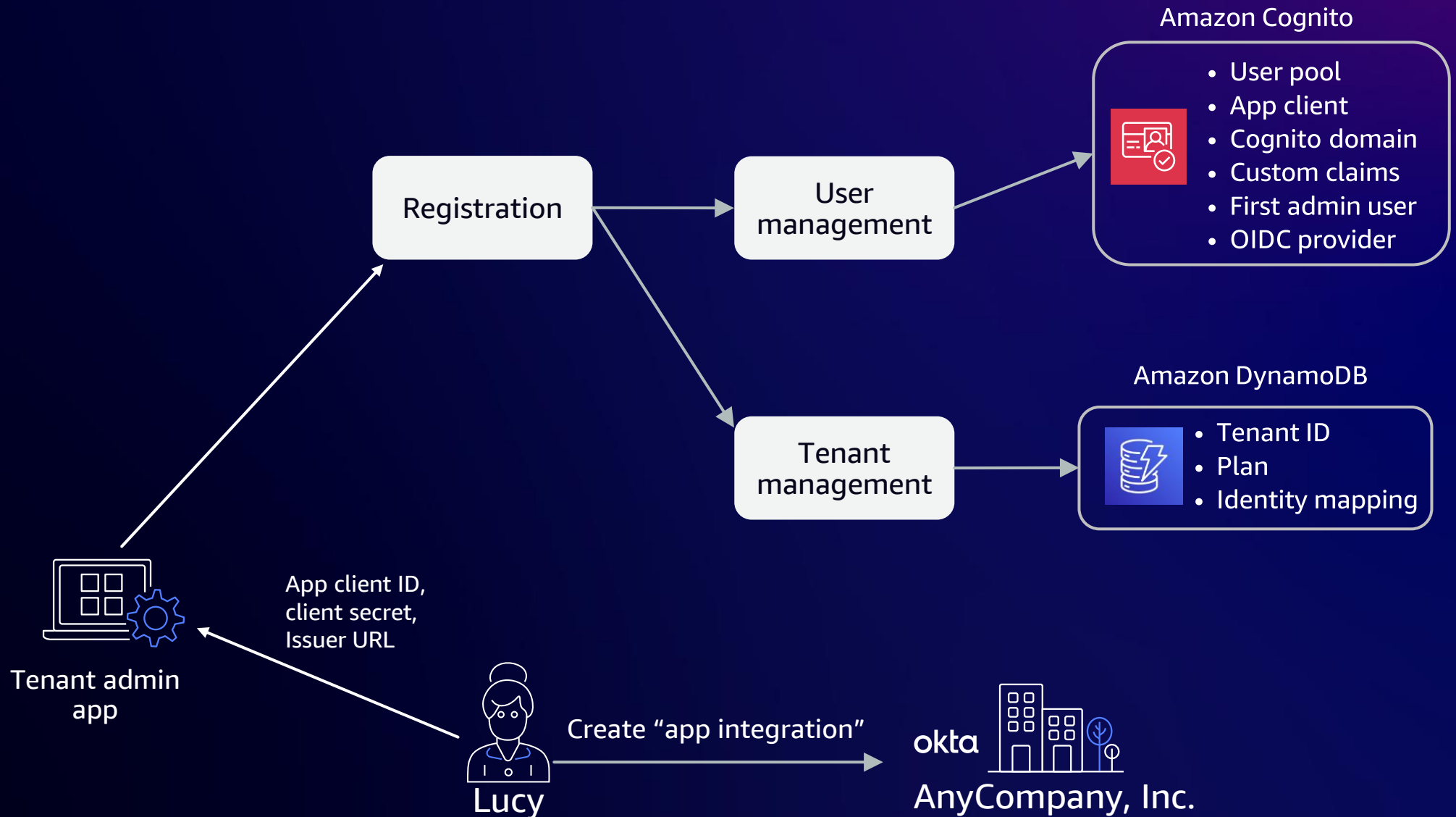
Tenant onboarding automation with federation



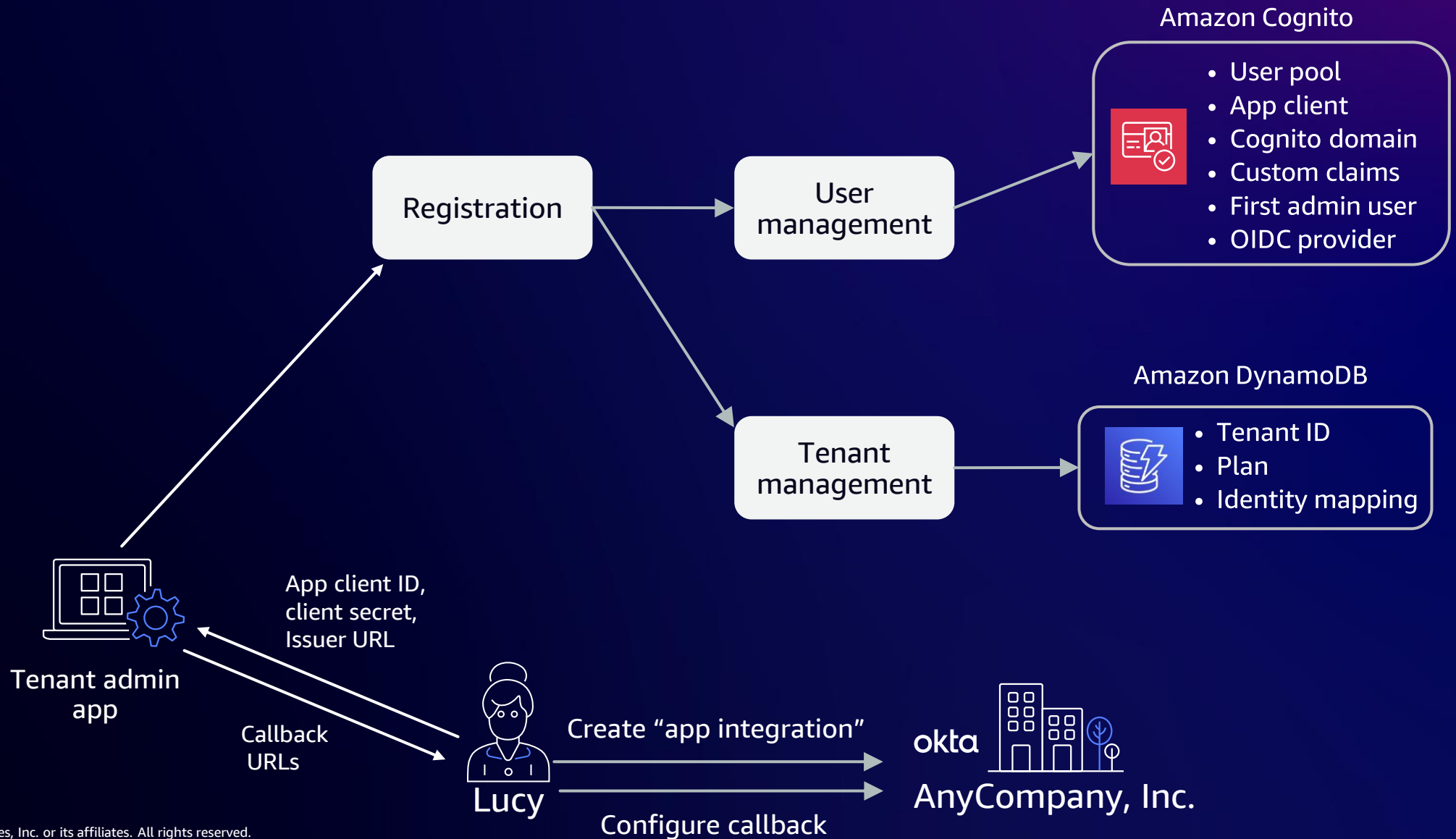
Tenant onboarding automation with federation



Tenant onboarding automation with federation



Tenant onboarding automation with federation



Federated Amazon Cognito's view

User pool

- Shadow(Proxy) user directory
- Federated users added to auto-created group corresponding to federated IdP
- Manages mapping of attributes from federated IdP

Federated
identity
provider

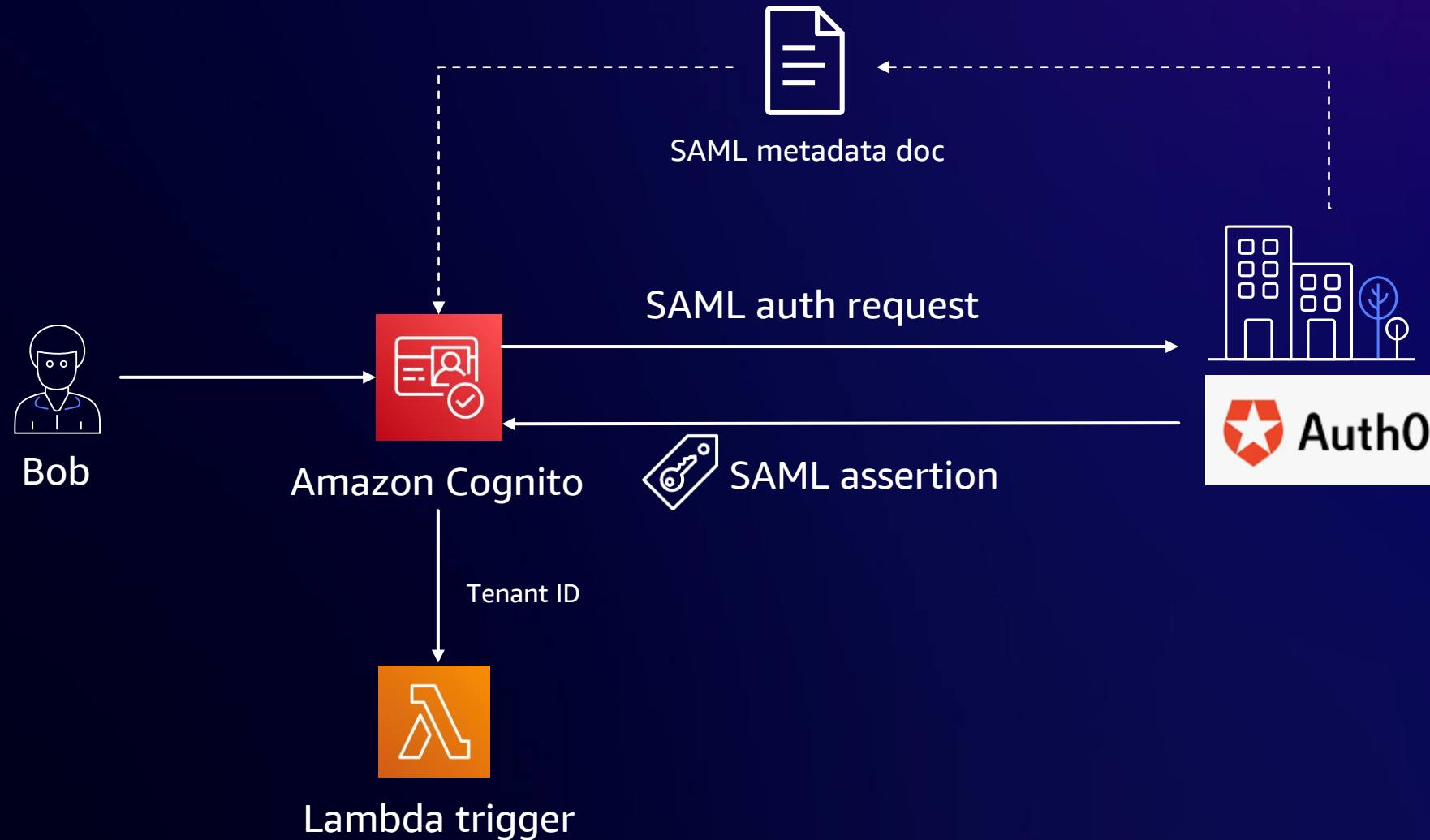
User pool "app" client

- Define authentication flows/settings
- Configure attribute access
- Token lifecycle
- Hosted UI definition/customization

"Cognito" domain

- Provides access to "hosted UI"
- Sets up OAuth2.0 standard endpoints

Amazon Cognito federation with SAML 2.0



Wrapping up

- Amazon Cognito is a federation hub
- User pools are still at the center of the identity experience
- Resolving tenants to user pools is still required
- Lambda triggers for injecting the “tenant context”
- A single onboarding process that covers internal and external SaaS identity

Additional SaaS sessions

Breakout sessions

- SAS305 – SaaS architecture pitfalls: Lessons from the field
- SAS304 – SaaS deep dive: Inside a scalable, efficient multi-tenant architecture
- SAS406 – SaaS DevOps deep dive: Automating multi-tenant deployments
- SAS306 – SaaS meets AI/ML & generative AI: Multi-tenant patterns & strategies
- SAS308 – SaaS anywhere: Designing distributed multi-tenant architectures
- ARC311 – Building cost-optimized multi-tenant SaaS architectures
- BWP301 – SaaS operations in action: Buy with Prime
- CON313 – Deploying multi-tenant SaaS applications on Amazon ECS and AWS Fargate

Additional SaaS sessions

Chalk talks

- SAS405-R – Federating identity in multi-tenant SaaS environments
- SAS404 – Designing and implementing a SaaS cost-per-tenant strategy
- SAS307 – SaaS tenant isolation: Patterns and strategies
- SAS302-R – A look inside SaaS networking, routing, and integration patterns
- SAS408 – Chaos and SaaS: Pushing your multi-tenant architecture
- SVS320 – Building multi-tenant applications with AWS Lambda and AWS Fargate
- ARC205 – SaaS jump-start: Exploring core multi-tenant strategies
- BIZ207 – Integrate your SaaS application or security tool with AWS AppFabric

Workshops

- SAS401-R – SaaS microservices deep dive: Multi-tenancy meets microservices
- SAS402 – SaaS multi-tenancy meets Amazon EKS: A hands-on deep dive
- SAS403 – SaaS survivor: Building a rich multi-tenant operations experience
- SAS301 – SaaS and AI/ML: Inside a multi-tenant AI/ML solution
- PEX401 – Building a multi-tenant SaaS control plane: An inside look

Additional SaaS sessions

Builder session

- SAS407 – Kubernetes DevOps tooling meets SaaS: Automating tenant provisioning

Business session

- PEX205 – Benchmarking your SaaS business on AWS

Explore resources from AWS SaaS Factory

BUILD AND SCALE YOUR SAAS BUSINESS WITH AWS



Visit **SaaS on AWS**

Access training, proven
best practices, and
solutions



Find **domain experts**

Find SaaS consulting
partners for design
and build support



Grow **your business**

Join the AWS
Partner Network for
additional benefits



[AWS.AMAZON.COM/SAAS](https://aws.amazon.com/saas)



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Thank you!

Toby Buckley

tobuck@amazon.com

Dhammika Sriyananda

sriyana@amazon.com



Please complete the session
survey in the mobile app